

Guide du projet

Catalogue de livres : extraction Playwright, PostgreSQL, API FastAPI, Docker

Sommaire

1. Présentation
2. Architecture
3. Chaîne de traitement
4. Modèle de données
5. API
6. Cycle de vie d'une extraction
7. Organisation du code
8. Configuration
9. Tests
10. Docker
11. Choix techniques et limites

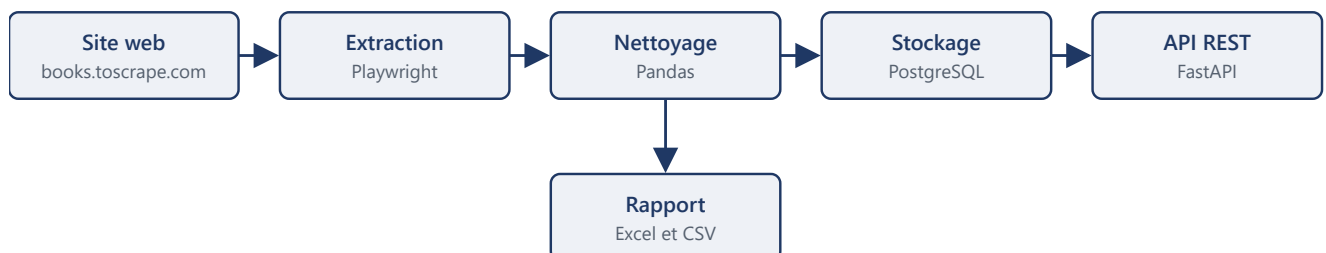
1. Présentation

Ce projet extrait le catalogue d'un site web, le stocke dans une base PostgreSQL et l'expose par une API REST. Il est lancé entièrement avec Docker.

Le site extrait est books.toscrape.com, un site d'entraînement public conçu pour cet usage. Une extraction complète parcourt 50 catégories et enregistre 1 000 livres : titre, prix, note, disponibilité et adresse de la page.

Besoin	Outil
Piloter un navigateur et lire les pages	Playwright
Nettoyer les données	Pandas
Stocker les données	PostgreSQL 16, SQLAlchemy 2, migrations Alembic
Exposer les données	FastAPI, Pydantic, Uvicorn
Produire un rapport	OpenPyXL (Excel) et CSV
Tester	pytest, collection Postman
Lancer et livrer	Docker, Docker Compose, GitHub Actions

2. Architecture



Chaîne de traitement : du site web jusqu'à l'API et au rapport.

Docker Compose décrit quatre services. Deux servent l'application, deux ne démarrent que pour les tests.

Service	Rôle	Accès depuis le poste
db	PostgreSQL de l'application. Les données sont conservées dans le volume <code>donnees</code> .	<code>localhost:5432</code>
api	Applique les migrations, puis lance l'API. Contient aussi Chromium pour les extractions.	<code>http://localhost:8000</code>
db-tests	PostgreSQL jetable, en mémoire, réservé aux tests.	aucun
tests	Lance pytest dans une image qui ajoute les outils de test.	aucun

3. Chaîne de traitement

Extraction

La classe `ExtracteurCatalogue` (`extraction.py`) ouvre la page d'accueil, relève la liste des catégories, puis parcourt chacune d'elles page par page en suivant le bouton « next ». Chaque page est lue en un seul appel exécuté dans le navigateur. Les images sont bloquées pour accélérer le parcours.

Nettoyage

La fonction `nettoyer` (`traitement.py`) transforme les textes bruts en colonnes typées : « £51.77 » devient le nombre 51.77, la note écrite en toutes lettres devient un chiffre de 1 à 5, la disponibilité devient un booléen. Les doublons sont retirés d'après l'adresse du livre.

Stockage

La fonction `enregistrer_livres` (`app/depot.py`) crée les catégories manquantes, puis écrit les livres par `INSERT ... ON CONFLICT (url) DO UPDATE`. Un livre déjà connu est mis à jour : relancer une extraction ne crée aucun doublon.

Rapport

La ligne de commande `main.py` génère en plus un classeur Excel (`export_excel.py`) avec une feuille d'indicateurs par catégorie, un graphique et la liste complète des livres, ainsi qu'un fichier CSV.

4. Modèle de données

Table `categories`

Colonne	Type	Remarque
<code>id</code>	entier	clé primaire
<code>nom</code>	texte (100)	unique

Table `livres`

Colonne	Type	Remarque
<code>id</code>	entier	clé primaire

Colonne	Type	Remarque
<code>categorie_id</code>	entier	clé étrangère vers <code>categories</code> , indexée
<code>titre</code>	texte	
<code>prix_gbp</code>	numérique (8,2)	prix en livres sterling
<code>note</code>	petit entier	de 1 à 5 ; vide si la note est inconnue
<code>en_stock</code>	booléen	
<code>url</code>	texte	unique ; identifie le livre d'une extraction à l'autre
<code>extraction_id</code>	entier	dernière extraction qui a écrit la ligne
<code>maj_le</code>	date et heure	dernière mise à jour

Table `extractions`

Colonne	Type	Remarque
<code>id</code>	entier	clé primaire
<code>statut</code>	texte (20)	<code>en_cours</code> , <code>terminee</code> ou <code>echouee</code>
<code>debut</code> , <code>fin</code>	date et heure	<code>fin</code> est vide tant que l'extraction tourne
<code>nb_livres</code>	entier	nombre de livres écrits
<code>erreur</code>	texte	message en cas d'échec

Un index unique partiel sur `extractions` (`statut = 'en_cours'`) interdit deux extractions simultanées. Le schéma est créé par la migration Alembic `migrations/versions/0001_tables_initiales.py` .

5. API

La documentation interactive est générée par FastAPI sur `/docs` (Swagger) et `/redoc` .

Méthode	Route	Rôle
GET	<code>/sante</code>	vérifie que l'API répond et joint la base
GET	<code>/livres</code>	liste paginée, filtrée et triée
GET	<code>/livres/{id}</code>	un livre
GET	<code>/categories</code>	catégories avec leur nombre de livres
GET	<code>/categories/indicateurs</code>	nombre de livres, prix moyen, minimum, maximum et note moyenne par catégorie
POST	<code>/extractions</code>	lance une extraction en tâche de fond (clé d'API exigée)
GET	<code>/extractions</code>	les 20 dernières extractions
GET	<code>/extractions/{id}</code>	état d'une extraction

Paramètres de GET /livres

Paramètre	Effet
categorie	nom exact de la catégorie, sans tenir compte de la casse
prix_min , prix_max	bornes de prix
note_min	note minimale, de 1 à 5
en_stock	true ou false
q	texte recherché dans le titre
tri	titre , prix_gbp ou note ; préfixe - pour un tri décroissant
page , taille	pagination : 20 livres par page par défaut, 100 au plus

Codes de réponse

Code	Signification
200	lecture réussie
202	extraction acceptée ; elle continue en arrière-plan
401	clé d'API absente ou invalide (en-tête X-API-Key)
404	livre ou extraction introuvable
409	une extraction est déjà en cours
422	paramètre invalide ; la réponse désigne le paramètre en cause
503	base de données injoignable

6. Cycle de vie d'une extraction

1. POST /extractions crée une ligne au statut en_cours et répond 202 immédiatement.
2. L'extraction tourne en tâche de fond dans le processus de l'API.
3. En cas de succès, les livres sont écrits et le statut passe à terminée , avec le nombre de livres.
4. En cas d'échec, aucun livre de cette extraction n'est conservé ; le statut passe à echouee et le message d'erreur est enregistré.

Une seconde demande pendant qu'une extraction tourne reçoit 409 : c'est l'index unique de la base qui refuse la ligne, même si deux demandes arrivent au même instant.

Si l'API s'arrête pendant une extraction, celle-ci est marquée echouee au démarrage suivant. Le verrou est ainsi toujours libéré.

7. Organisation du code

Fichier	Rôle
<code>extraction.py</code>	classe <code>ExtracteurCatalogue</code> : navigation, pagination, lecture des pages
<code>traitement.py</code>	nettoyage et indicateurs avec Pandas
<code>export_excel.py</code>	rapport Excel avec OpenPyXL
<code>main.py</code>	ligne de commande : extraction, écriture en base, rapport
<code>app/main.py</code>	application FastAPI
<code>app/routes/</code>	routes : livres, catégories, extractions, santé
<code>app/schemas.py</code>	schémas Pydantic des paramètres et des réponses
<code>app/models.py</code>	tables SQLAlchemy
<code>app/depot.py</code>	tout le SQL : écriture des livres, requêtes de lecture
<code>app/service_extraction.py</code>	déroulement d'une extraction et suivi de son statut
<code>app/config.py</code> , <code>app/db.py</code>	réglages et connexion à la base
<code>migrations/</code>	migrations Alembic
<code>tests/</code>	tests pytest et mini-site local <code>tests/site/</code>
<code>tests/postman/</code>	collection Postman
<code>Dockerfile</code> , <code>docker-compose.yml</code>	images et services
<code>.github/workflows/tests.yml</code>	intégration continue

8. Configuration

Les réglages sont lus dans les variables d'environnement. Avec Docker, ils se placent dans un fichier `.env` à la racine du projet (modèle : `.env.example`). Ce fichier n'est pas publié sur Git.

Variable	Rôle	Valeur par défaut
<code>CLE_API</code>	clé exigée pour lancer une extraction	<code>changez-moi</code>
<code>POSTGRES_PASSWORD</code>	mot de passe de la base, lu à sa création	<code>catalogue</code>
<code>DATABASE_URL</code>	adresse de la base ; réglée par Docker Compose	base locale <code>catalogue</code>
<code>NAVIGATEUR</code>	<code>chrome</code> , <code>msedge</code> ou <code>chromium</code>	<code>chromium</code> dans Docker, <code>chrome</code> sinon
<code>NAVIGATEUR_TESTS</code>	navigateur utilisé par les tests	<code>chrome</code>

9. Tests

Tests pytest : 52 tests

```
docker compose --profile tests run --rm tests
```

Fichier	Tests	Ce qui est vérifié
test_extraction.py	4	navigateur réel sur le mini-site local : catégories, pagination, champs
test_traitement.py	6	conversion des prix, notes, stock, doublons, indicateurs
test_export_excel.py	3	feuilles, lignes, mise en forme du classeur
test_depot.py	4	écriture, mise à jour sans doublon, concordance migrations et modèles
test_api.py	25	santé, liste, pagination, filtres, tris, erreurs, catégories, indicateurs
test_api_extractions.py	10	clé d'API, extraction complète, verrou, échec, reprise après arrêt

Les tests n'accèdent pas au réseau. Ceux de la base et de l'API tournent sur un vrai PostgreSQL, dont les tables sont créées par les migrations. Comme ils vident les tables, ils refusent toute base dont le nom ne finit pas par `_test`.

Collection Postman : 27 requêtes, 80 vérifications

La collection `tests/postman/catalogue.postman_collection.json` teste l'API en fonctionnement, y compris le cycle complet d'une extraction. Elle se lance dans Postman ou avec Newman.

Intégration continue

À chaque envoi de code sur GitHub, le workflow GitHub Actions lance la même commande de test que ci-dessus, puis construit l'image de production.

10. Docker

Le `Dockerfile` produit deux images à partir d'une base commune (Python 3.12, dépendances, Chromium) :

- `prod`, la cible par défaut : applique les migrations puis lance Uvicorn sur le port 8000.
- `tests` : ajoute pytest et le dossier `tests/`.

Dans les deux cas, l'application tourne sous un utilisateur non administrateur.

Action	Commande
Démarrer l'application	<code>docker compose up -d --build</code>
Lancer une extraction avec rapport	<code>docker compose run --rm api python main.py</code>
Voir les journaux de l'API	<code>docker compose logs -f api</code>
Ouvrir une console SQL	<code>docker compose exec db psql -U catalogue -d catalogue</code>

Action	Commande
Lancer les tests	<code>docker compose --profile tests run --rm tests</code>
Arrêter	<code>docker compose down</code>
Arrêter et effacer la base	<code>docker compose down -v</code>

11. Choix techniques et limites

- **SQLAlchemy synchrone.** Playwright est utilisé en mode synchrone et le volume est faible ; l'asynchrone ajouterait du code sans gain mesurable.
- **Tâche de fond FastAPI** plutôt qu'une file de messages. C'est suffisant pour une extraction à la fois, mais l'extraction vit dans le processus de l'API : elle est perdue si celui-ci s'arrête, et l'API doit tourner avec un seul processus.
- **Verrou porté par la base.** L'index unique partiel évite tout verrou applicatif.
- **Tests sur un vrai PostgreSQL.** L'écriture `ON CONFLICT` et l'index partiel sont propres à PostgreSQL ; une base de substitution ne les testerait pas.
- **Sécurité.** Seul le lancement d'une extraction exige une clé. Les lectures sont ouvertes : l'API n'est pas prévue pour être exposée telle quelle sur Internet.